Home Control

Meine Projektbeschreibung | Mario Kujoth

1 Warum ich Home Control gebaut habe

Meine Geräte waren auf mehrere Apps verteilt. Wenn ich wissen wollte, wie warm es im Wohnzimmer ist, ob die Flurtür offen steht oder was die Kamera zuletzt aufgenommen hat, musste ich zwischen den Apps wechseln. Deshalb habe ich im Rahmen meiner Umschulung Home Control gebaut. Ich wollte die wichtigsten Informationen pro Raum sehen und ausgewählte Geräte von einer Stelle aus bedienen.

Angefangen habe ich mit Räumen, Geräten und einem Verlauf. Danach kamen die Verbindungen zu den Herstellern, das Schalten und die Türregel dazu. Für den Zugriff vom Handy habe ich das System auf einen OVH-Server umgezogen. Über die eigene Schnittstelle habe ich auch meinen Muju-Agenten angebunden.

> **Mein Ziel:** Auf einen Blick sehen, was zu Hause passiert. Bei einem Schaltbefehl möchte ich wissen, ob das Gerät den neuen Zustand wirklich gemeldet hat.

Seite	Thema
2	Wie ich die Anwendung aufgebaut habe
3	Welche Schnittstellen ich selbst gebaut habe
4	Welche externen APIs und Dienste ich nutze
5	Wie Schalten, Raumklima und Messwerte funktionieren
6	Wie Türkontakt, Blink und Alarmclips zusammenspielen
7	Warum ich auf OVH umgezogen bin
8	Wie mein Muju-Agent angebunden ist und was ich geprüft habe

Wie ich gearbeitet habe

Ich habe festgelegt, welche Geräte eingebunden werden und welche Aktionen die App ausführen darf. Beim Programmieren, Prüfen und Dokumentieren habe ich Claude und Codex als Werkzeuge eingesetzt. Die Ergebnisse an meinen Geräten habe ich in die weitere Entwicklung einbezogen.

2 Wie ich die Anwendung aufgebaut habe

Ich habe die Oberfläche und die Logik im Server getrennt. Die Oberfläche soll Informationen gut darstellen und Eingaben aufnehmen. Im Backend liegen die Regeln, die Anmeldung und die Verbindungen zu den Geräten. So kann ich zum Beispiel eine Geräteanbindung ändern, ohne dafür die ganze Oberfläche umzubauen.

Baustein	Warum ich ihn verwende
React, TypeScript und Vite	Mit React kann ich Raumkarten, Schalter und Statusanzeigen wiederverwenden. TypeScript prüft, ob die Daten beim Aufruf zur erwarteten Form passen. Vite baut die Web-App.
C# und ASP.NET Core 10	Hier prüfe ich die Rechte, verarbeite Anfragen und rufe die passende Geräteanbindung auf. Im Betrieb liefert das Backend auch die gebaute Web-App aus. Für beides genügt damit eine Adresse.
EF Core und SQLite	Ich betreibe das System als einzelne Instanz. SQLite hält den Betrieb einfach. EF Core ordnet die C#-Modelle den Tabellen zu und verwaltet Änderungen am Schema über Migrationen.

Welche Daten ich speichere

In der Hausdatenbank liegen unter anderem Räume, Gerätezuordnungen und Ereignisse. Die Klimawerte liegen in einer eigenen SQLite-Datei. So kann ich beim Umzug beide Bestände gezielt sichern und übernehmen. Für Benutzerkonten verwende ich ASP.NET Core Identity.

Wie Änderungen im Browser ankommen

Über REST fragt der Browser gezielt Daten ab oder sendet eine Aktion. Für Statusänderungen nutze ich zusätzlich Server-Sent Events (SSE). Dabei hält der Browser einen Datenstrom vom Backend offen. Das ist der Weg **vom Backend zum Browser**. Die Tuya-Cloud fragt das Backend weiterhin in Intervallen ab.

> Der Ablauf einer Aktion führt von der Oberfläche über die API zum passenden Gerätedienst. Die Oberfläche übernimmt den neuen Schaltzustand nach der Rückmeldung.

3 Welche Schnittstellen ich selbst gebaut habe

In meinem Backend habe ich **eine eigene REST-API** gebaut. Ich teile ihre Endpunkte hier in vier Aufgabenbereiche ein. Ein Endpunkt ist eine Adresse für eine bestimmte Abfrage oder Aktion. Die Übersicht zeigt Beispiele direkt aus meinem Code.

Aufgabenbereich	Beispiel aus meiner API	Wofür ich ihn brauche
1. Hausdaten und Status	**GET /api/rooms** GET /api/devices GET /api/status	Räume, Geräte und den Verbindungsstatus einheitlich an die Oberfläche liefern. Dazu kommen Verlauf und Tagesauswertungen.
2. Anmeldung und Rechte	**POST /api/auth/login** POST /api/auth/refresh	Eine Sitzung eröffnen und erneuern. Die Rechte prüfe ich im Backend, damit ein direkter Aufruf dieselben Regeln einhält wie ein Klick in der Oberfläche.
3. Android-Bridge	**POST /api/climate/bridge/readings** POST /api/google-home/bridge/poll	Klimawerte annehmen. Muju Home holt über den zweiten Endpunkt Befehle ab und meldet das Ergebnis über einen eigenen Rückkanal.
4. Steuerung und Alarm	**POST /api/tuya/devices/{id}/switch** GET /api/alarm/clips	Erlaubte Schaltaktionen ausführen und Alarmdaten an berechtigte Benutzer liefern. Für Google-Home-Schalter und Szenarien gibt es eigene Endpunkte im selben Backend.

Warum ich die API so aufgebaut habe

Der Browser und mein Muju-Agent können dieselben vorgesehenen Lese-Endpunkte nutzen. Die Herstellerdetails bleiben in den jeweiligen Anbindungen. Dadurch arbeiten das Dashboard und der Agent mit einer gemeinsamen Sicht auf meine Geräte. Den Eingang für die Android-Bridge habe ich separat abgesichert, weil sich das Handy mit einem eigenen Bridge-Schlüssel meldet.

Im Entwicklungsmodus kann ich die API-Beschreibung in Scalar öffnen und sehen, welche Anfragen ein Endpunkt erwartet und welche Antworten möglich sind. Im Betrieb prüft das Backend die Anmeldung und die Rolle am jeweiligen Endpunkt.

4 Welche externen Schnittstellen ich nutze

Die vier zentralen Anbindungen liefern Daten oder Funktionen von außen. Ich habe die jeweiligen Schnittstellen in Home Control eingebunden und ihre Ergebnisse für meine Oberfläche aufbereitet.

Anbindung	Was ich abrufe oder auslöse	Warum ich sie eingebunden habe
Tuya Cloud API	Geräteliste, Online-Status, verfügbare Messwerte und Ein/Aus bei geeigneten Geräten.	Viele meiner Geräte sind bereits in Tuya eingerichtet. Ich kann ihre Zustände zentral anzeigen und unterstützte Geräte gezielt bedienen.
Google Home APIs	Muju Home nutzt das Android-SDK für Raumklima, Türkontakt und verknüpfte Schalter.	Damit bekomme ich Daten und Schaltmöglichkeiten für Geräte, die ich über mein Tuya-Projekt nicht erreichen konnte.
Blink	Kameraliste, Vorschaubilder, verfügbare Clips und das Scharfschalten eines ausgewählten Systems.	Ich wollte Kameras und Türereignisse in derselben Anwendung sehen. Dafür nutze ich einen eigenen Client für die inoffizielle Blink-Schnittstelle.
Open-Meteo	Außentemperatur und Wetterdaten für die Übersicht.	Das Dashboard soll neben den Werten aus den Räumen auch das Wetter draußen zeigen. Die Abfrage wird kurz zwischengespeichert.

Welche weiteren Messwege ich vorbereitet habe

Für geeignete Shelly-Geräte kann ich Leistung und Zählerstand über die lokale HTTP-RPC-Schnittstelle abfragen. Für kompatible Bluetooth-Klimasensoren gibt es unter Linux eine eigene Bridge. Sie reicht ihre Werte über den vorhandenen Klima-Endpunkt weiter.

> **So zähle ich die Schnittstellen:** Eine selbst gebaute REST-API mit mehreren Bereichen und vier zentrale externe Anbindungen. Shelly und Bluetooth sind zusätzliche lokale Messwege.

5 Wie Schalten und Messwerte funktionieren

Warum ich beim Schalten eine Bestätigung brauche

Bei Tuya ordne ich einem Gerät in Home Control das passende Cloud-Gerät zu. Meine API prüft vor dem Befehl die Admin-Rolle, die Erreichbarkeit und den unterstützten Schaltpunkt. Nach dem Befehl liest sie den Zustand erneut. So sehe ich, ob das Gerät den Wechsel tatsächlich gemeldet hat.

Warum meine Stehlampe einen anderen Weg nimmt

Die Steckdose der Wohnzimmer-Stehlampe liegt in Lidl Home. Mein Tuya-Projekt hatte dafür keine Berechtigung, über Google Home war sie erreichbar. Deshalb habe ich Muju Home als Android-Bridge genutzt. Die App kennt die zugeordneten Google-Home-Geräte und fragt das Backend ungefähr alle 1,5 Sekunden nach offenen Befehlen.

1. Home Control legt einen Schaltbefehl für das verknüpfte Gerät bereit.
2. Muju Home holt ihn innerhalb von vier Sekunden ab und schaltet über Google Home.
3. Die App meldet den beobachteten Zustand zurück. Abgelaufene Befehle bleiben verworfen.

Wie ich Klima, Strom und Akku darstelle

Muju Home sendet Temperatur und Luftfeuchte an meine Klima-Schnittstelle. In der Oberfläche ordne ich den Sensor einem Raum zu und zeige die Zeit des letzten Empfangs an. Ältere Werte sind gekennzeichnet.

Watt zeigt die aktuelle Leistung. Aus aufeinanderfolgenden Wattmessungen schätze ich den Verbrauch in Kilowattstunden und kennzeichne ihn entsprechend. Gemeldete Shelly-Zählerstände stehen separat. Fehlende Strom- oder Akkuwerte lasse ich offen. Die PC-Steckdose behandle ich ausschließlich als Messpunkt.

6 Wie Türkontakt, Blink und Alarm zusammenspielen

Ich wollte aus einem Türsignal eine praktische Aktion machen. Meine Regel schaltet deshalb das ausgewählte Blink-System scharf, wenn die Flurtür frisch von Zu auf Auf wechselt. Die Regel läuft im Backend und damit auch bei geschlossenem Browser.

Schritt	Was im System passiert
Türkontakt	Muju Home sendet den Zustand. Mein Backend erkennt einen frischen Wechsel. Wiederholte oder veraltete Meldungen lösen die Regel nicht aus.
Regel	Eine aktivierte Regel sendet den Scharfschaltbefehl an das ausgewählte Blink-System. Neue Regeln sind zunächst ausgeschaltet.
Rückmeldung	Das Backend liest den Zustand erneut. Der Verlauf zeigt Bestätigt erst, wenn Blink Scharf meldet.

Warum es eine Alarmseite gibt

Dort sehe ich die Ergebnisse meiner Regeln und die verfügbaren Clips vom USB-Speicher des Blink-Sync-Moduls. Home Control markiert einen Clip als zeitlich passend, wenn er zwischen 30 Sekunden vor und drei Minuten nach dem Türereignis aufgenommen wurde. Die Markierung beschreibt den zeitlichen Zusammenhang. Die Ursache der Aufnahme bleibt offen.

Videos werden erst beim Abspielen über das Backend geladen. Ein echter Testclip brach im Browser wegen eines leeren Videopakets ab. Deshalb verarbeitet ffmpeg die heruntergeladene Kopie vor der Ausgabe, ohne Bild und Ton neu zu kodieren. Die Originaldatei bleibt auf dem Stick erhalten.

Was ich an der Kamera sehen kann

Die Kameraansicht zeigt das letzte Standbild mit Aufnahmezeit. Als Admin kann ich ein neues Bild anfordern. Für Livebilder gibt es eine Probe, die den Empfang von Videodaten prüft. Die Live-Wiedergabe im Browser ist noch offen.

7 Warum ich das System auf OVH betreibe

Anfangs lief Home Control auf meinem Linux-PC. Ich wollte die Oberfläche auch unterwegs aufrufen können und habe sie deshalb auf OVH umgezogen. Ich habe Hausdaten, Klimadaten und die Data-Protection-Schlüssel gesichert und in die Docker-Volumes übernommen. Die Schlüssel braucht der Server unter anderem zum Lesen des verschlüsselten Blink-Tokens.

Teil	Meine Entscheidung
Docker-Image	Node baut die Oberfläche. Das .NET SDK veröffentlicht die API. Das Laufzeit-Image enthält die App und ffmpeg für die Clips.
Container und Volumes	Compose startet Home Control und den Sensor-Gateway. Die Daten und Schlüssel liegen in Volumes, damit ein neues Image sie weiterverwenden kann.
Privater Zugriff	Tailscale stellt den HTTPS-Zugang für die Web-App und einen getrennten Zugang für die Android-Bridge bereit. Auf dem iPhone öffne ich die Web-App über Tailscale.

Was ich beim Umzug anpassen musste

Beim ersten Start konnte der App-Benutzer im Container die Konfiguration nicht lesen. Die Dateien waren beim Entpacken zu restriktiv geworden. Ich habe der App-Gruppe die nötigen Leserechte gegeben und den Start aus einem privat entpackten Archiv in die Docker-Prüfung aufgenommen.

Blink beantwortete dieselbe Testanfrage vom Heimanschluss mit HTTP 401 und von OVH mit HTTP 406. Deshalb leite ich nur die Blink-HTTP-Anfragen über einen privaten Proxy am Heimanschluss. Für diesen Proxy muss der Linux-PC aktuell laufen. Die Android-Bridge habe ich zusätzlich für den direkten Weg per WLAN und Tailscale eingerichtet. Dabei bleibt Muju Home auf dem Android-Handy geöffnet.

> Die Web-App und die direkte Tuya-Anbindung laufen auf OVH. Google Home braucht die Android-Bridge. Blink braucht in meinem Aufbau den Heimproxy.

8 Wie ich meinen Muju-Agenten angebunden habe

Ich habe Home Control mit meinem Muju-Agenten verbunden. Der Agent läuft über n8n und nutzt meine REST-API als Datenquelle. Damit kann er zum Beispiel den aktuellen Systemstatus über **GET /api/status** abrufen und die Antwort für eine Auskunft aufbereiten.

Schritt	So ist die Verbindung aufgebaut
n8n sendet eine Anfrage	Der Workflow ruft den vorgesehenen Home-Control-Endpunkt auf. Den Schlüssel speichere ich in n8n als Zugangsdaten.
Home Control prüft den Schlüssel	Der Workflow sendet ihn im Header X-Api-Key. Das Backend prüft ihn und gibt die für den Agenten freigegebenen Daten zurück.
Der Agent liest die Antwort	Der Schlüssel erlaubt lesende Zugriffe. Schaltbefehle und andere Admin-Aktionen brauchen eine Admin-Sitzung. Private Kamerainhalte haben zusätzliche Zugriffsvorgaben.

Ich habe den Agentenzugriff getrennt, weil der Agent für eine Statusauskunft weder mein Benutzerpasswort noch Schaltrechte braucht. Den API-Schlüssel und die Hersteller-Zugangsdaten halte ich aus dem Repository heraus.

Was ich praktisch geprüft habe

Auf OVH waren Tuya, Live-Klima und die Blink-Kameras erreichbar. Die Türregel wurde am echten System ausgelöst. Einen gespeicherten Clip konnte ich im Browser abspielen und spulen. Die Android-App für den direkten WLAN-Weg wurde installiert. Der vollständige Stehlampentest über Mobilfunk bei ausgeschaltetem Linux-PC ist der noch offene Abnahmeschritt.

Was ich aus dem Projekt mitnehme

Am meisten habe ich bei der Anbindung echter Geräte gelernt. Ein sauberer Build zeigt, dass der Code gebaut werden kann. Der Gerätetest zeigt, ob eine Meldung ankommt und eine Aktion wirklich ausgeführt wird. Für Vorführungen habe ich deshalb eine getrennte Demo mit Beispieldaten. Den Quellcode und die einzelnen Anleitungen habe ich auf GitHub dokumentiert.